

Phân tích đối chứng kiến trúc và hiệu năng: Standard RAG vs. GraphRAG

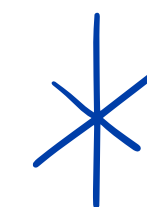


NHÓM 2



LLM & RAG là gì

- Định nghĩa: Retrieval-Augmented Generation (RAG) là kỹ thuật kết nối các mô hình ngôn ngữ lớn (LLM) với nguồn dữ liệu bên ngoài (tài liệu nội bộ, cơ sở dữ liệu thực tế).
- Mục tiêu: Cho phép AI trả lời dựa trên dữ liệu cập nhật và chính xác, thay vì chỉ dựa vào bộ nhớ đào tạo tĩnh.
- RAG giúp giảm thiểu ảo giác (hallucination) và cho phép trả lời câu hỏi theo miền chuyên biệt
- Cơ chế hiện tại: mã hóa đoạn văn → vector dày đặc → tìm kiếm cosine similarity → lấy Top-K





Standard RAG: Kiến trúc 3 giai đoạn

Giai đoạn 1 — Lập chỉ mục (Indexing):

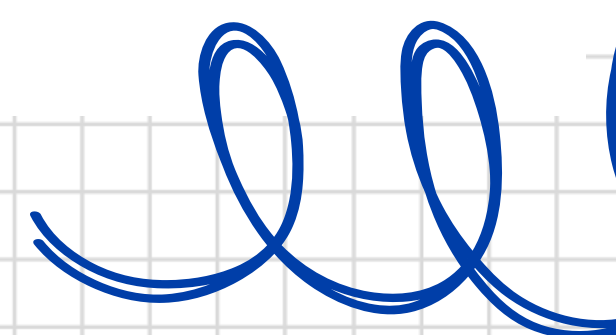
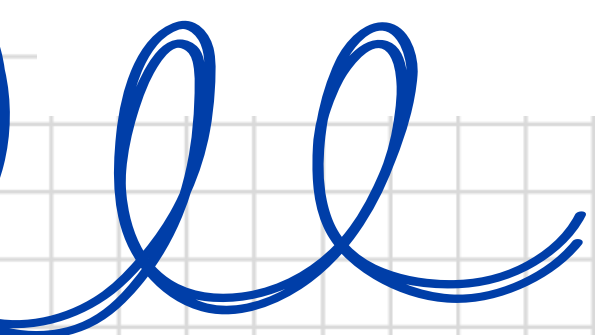
- Chia tài liệu thành các đoạn văn (chunks)
- Mã hóa qua mô hình embedding → vector dày đặc
- Lưu vào cơ sở dữ liệu vector (FAISS)

Giai đoạn 2 — Truy xuất (Retrieval):

- Nhúng câu truy vấn thành vector
- Tính độ tương đồng cosine với tất cả đoạn trong chỉ mục
- Trả về Top-K đoạn có điểm tương đồng cao nhất

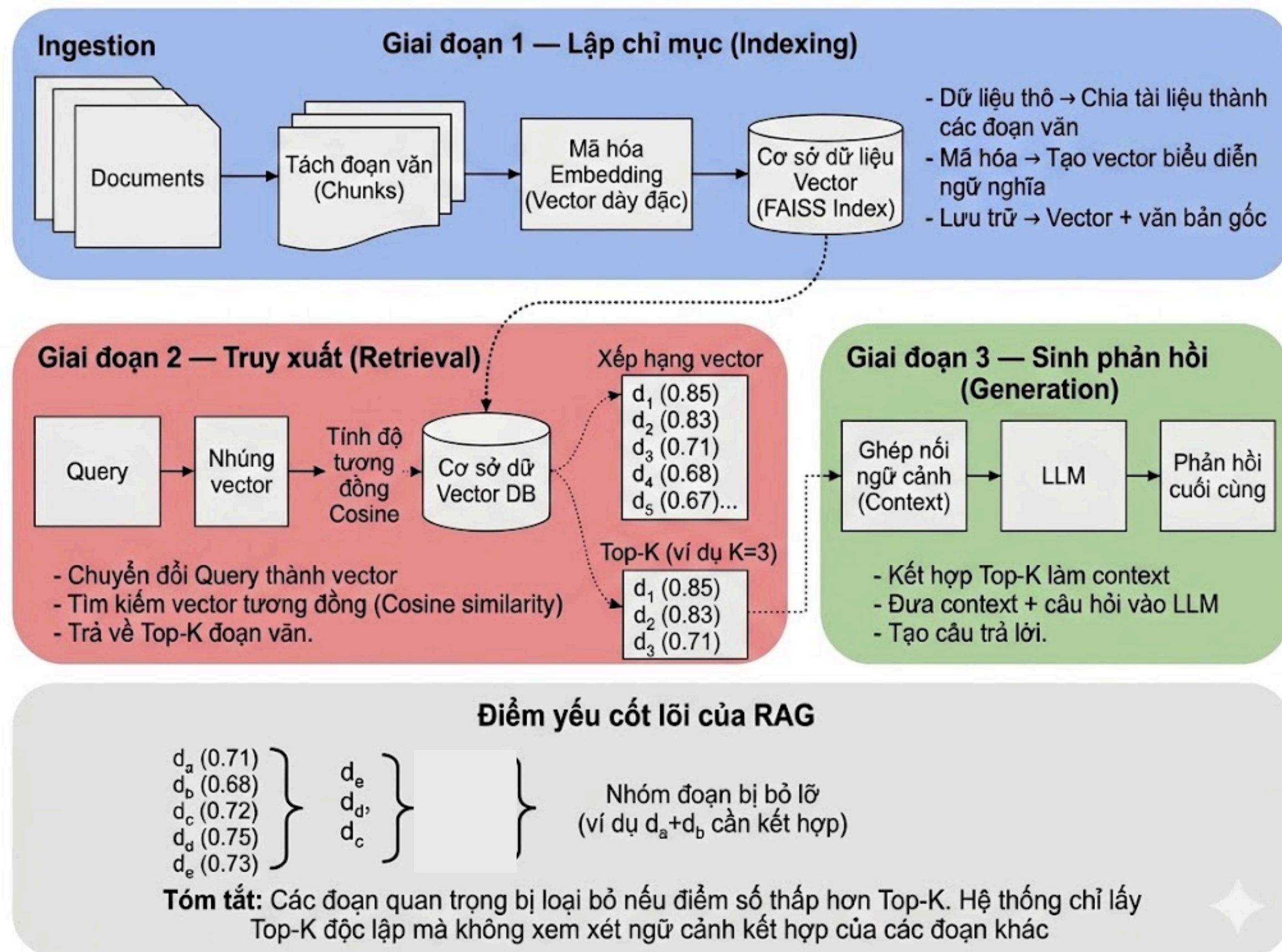
Giai đoạn 3 — Sinh phản hồi (Generation):

- Ghép K đoạn đã truy xuất làm ngữ cảnh (context)
- Đưa vào LLM kèm câu hỏi gốc
- LLM sinh câu trả lời cuối cùng



- Hạn chế của RAG cơ bản:
Chỉ tìm kiếm vector, dễ mất ngữ cảnh.
- Cải tiến (Advanced RAG):
Tối ưu bằng Hybrid Search, Re-ranking, Query Expansion.
- Đột phá (GraphRAG): Kết hợp Knowledge Graph để hiểu sâu mối quan hệ giữa các dữ liệu.

Kiến trúc RAG 3 giai đoạn: Quy trình hoạt động chi tiết



GraphRAG là gì

- Định nghĩa: GraphRAG là kỹ thuật mở rộng của RAG, kết hợp LLM với đồ thị tri thức (các thực thể và mối quan hệ giữa chúng).
- Mục tiêu: Giúp AI trả lời câu hỏi cần suy luận nhiều bước và hiểu mối liên kết ngầm giữa các thông tin nằm rải rác trong nhiều tài liệu.
- GraphRAG giúp khắc phục giới hạn trong câu hỏi truy vết quan hệ, tổng hợp xuyên suốt nhiều nguồn, và giải thích được kết quả nhờ đường đi trên đồ thị.
- Cơ chế hiện tại: Trích xuất thực thể & quan hệ → xây đồ thị tri thức → duyệt theo cạnh liên kết → tổng hợp nhiều bước suy luận → lấy Top-K



GraphRAG: Kiến trúc 5 giai đoạn



Giai đoạn 1 — Trích xuất đồ thị:

- LLM nhận diện thực thể (người, tổ chức, địa điểm, khái niệm) và quan hệ từ mỗi đoạn văn, tạo biểu diễn có cấu trúc (E, R).

Giai đoạn 2 — Tóm tắt mô tả:

- Nhiều mô tả của cùng một thực thể xuất hiện ở các đoạn khác nhau được hợp nhất thành một bản tóm tắt chuẩn nhất quán.

Giai đoạn 3 — Phát hiện cộng đồng:

- Thuật toán Leiden phân vùng đồ thị thực thể thành các cộng đồng gắn kết về mặt ngữ nghĩa.

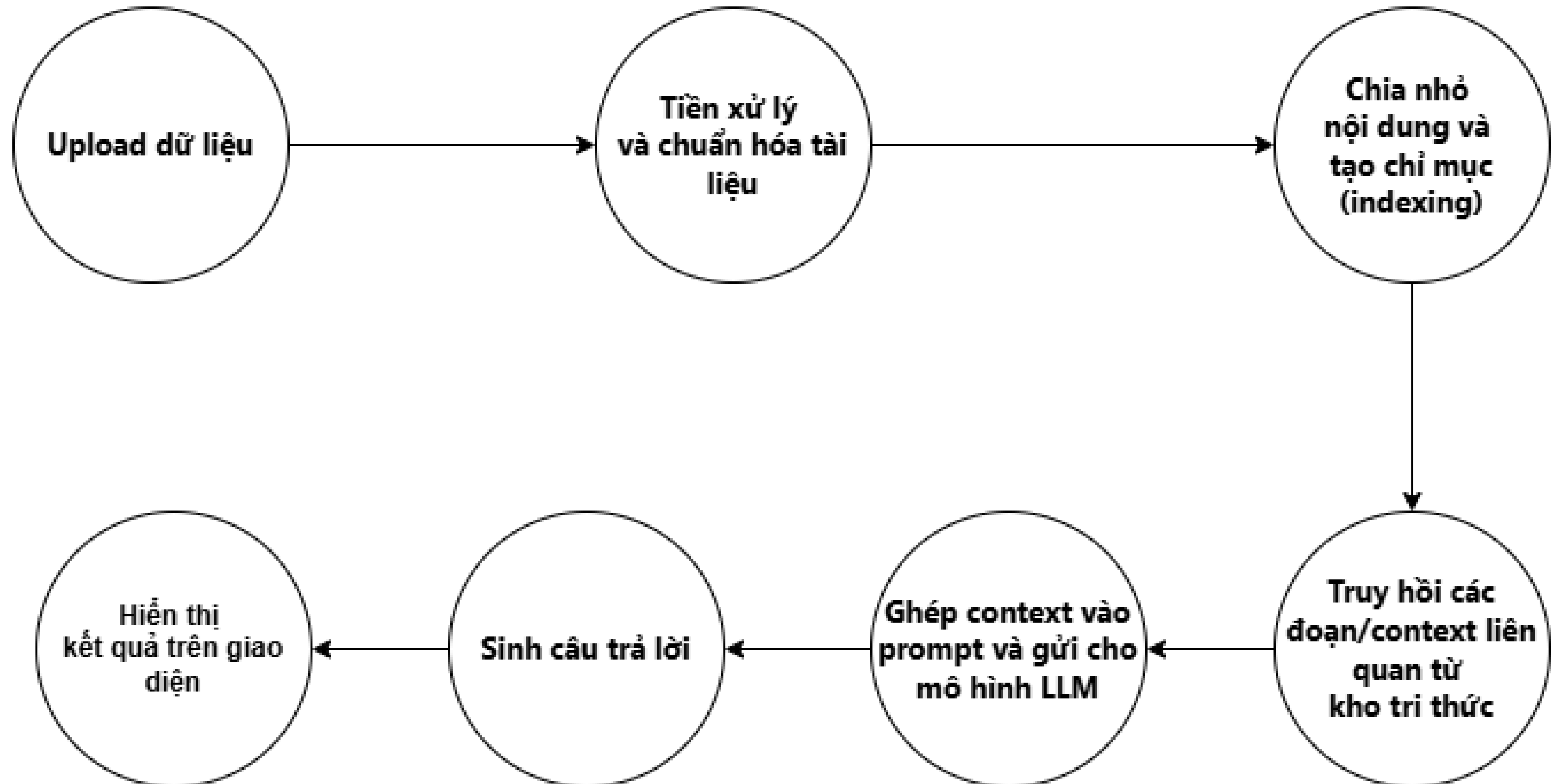
Giai đoạn 4 — Báo cáo cộng đồng:

- LLM tạo báo cáo tóm tắt bằng ngôn ngữ tự nhiên toàn diện cho mỗi cộng đồng.

Giai đoạn 5 — Tạo Embedding:

- Mô tả thực thể, đơn vị văn bản và báo cáo cộng đồng đều được nhúng để phục vụ truy xuất lai giữa vector và đồ thị.

GraphRAG





Các công nghệ sử dụng

- Ngôn ngữ: Python
- Backend: FastAPI, Uvicorn, Pydantic
- Giao diện: Streamlit
- Trực quan hóa đồ thị: streamlit-agraph, D3.js
- Mô hình AI/LLM: Groq, OpenAI API
- RAG/GraphRAG: Microsoft GraphRAG, LangChain
- Tìm kiếm vector: FAISS, ChromaDB
- Embedding: sentence-transformers, HuggingFace
- Xử lý tài liệu: PyMuPDF, python-docx
- Xử lý dữ liệu: Pandas, NumPy

Các chức năng chính của hệ thống gồm:

- Upload tài liệu: nhận file PDF, DOCX, TXT và các file văn bản khác.
- Tiền xử lý dữ liệu: trích xuất nội dung từ PDF/DOCX, chuyển về dạng text.
- Indexing dữ liệu: xây chỉ mục cho GraphRAG và Standard RAG.
- Chat hỏi đáp: người dùng đặt câu hỏi và hệ thống trả lời theo ngữ cảnh tài liệu.
- Truy vấn nâng cao:
 - Local Search
 - Global Search
- So sánh RAG và GraphRAG: hiển thị kết quả song song giữa hai phương pháp.
- Trực quan hóa knowledge graph: xem các entity và mối quan hệ giữa chúng.
- Quản lý domain/tài liệu: xem danh sách domain, trạng thái index, xóa hoặc re-index.
- Lưu lịch sử chat: quản lý các phiên hội thoại đã tạo.

Chiến lược truy vấn: Local Search vs. Global Search

Local Search — dành cho câu hỏi về thực thể cụ thể:

- Kết hợp tìm kiếm vector trên embedding đơn vị văn bản với duyệt đồ thị quanh các nút thực thể liên quan
- Ví dụ: "Thuật toán Leiden là gì?", "FastAPI được dùng để làm gì?"
- Độ trễ trung bình: 4,8 giây
- Độ phức tạp:
 $T_{\text{Local}} = O(K \cdot d) + O(\deg(v) \cdot d)$

Global Search — dành cho câu hỏi tổng hợp toàn cục:

- Đọc các Báo cáo Cộng đồng đã được tạo sẵn từ trước — không quét từng đoạn riêng lẻ
- Ví dụ: "Hệ thống có điểm mạnh và hạn chế gì?", "So sánh hai kiến trúc"
- Độ trễ trung bình: 12,3 giây
- Độ phức tạp:
 $T_{\text{Global}} = O(|C| \cdot L_r)$

Thiết lập thực nghiệm

Tập dữ liệu:

- 5 tài liệu PDF kỹ thuật
- 47 trang tổng cộng
- Nội dung: tài liệu học thuật về AI/NLP

Tập truy vấn — 30

câu hỏi:

- 15 truy vấn cụ thể (factoid): hỏi về định nghĩa, số liệu, sự thật cụ thể
- 15 truy vấn toàn cục (synthesis): đòi hỏi tổng hợp, so sánh, suy luận đa bước

Phương pháp đánh giá:

- 3 đánh giá viên độc lập, chấm theo thang Likert 1–5
- Độ tin cậy liên đánh giá: Cohen's $\kappa = 0,74$ → mức đồng thuận đáng kể

Các chỉ số đo lường:

- Accuracy: tỉ lệ câu trả lời có điểm ≥ 4 về độ chính xác
- Completeness: tỉ lệ câu trả lời có điểm ≥ 4 về độ đầy đủ
- Latency: thời gian phản hồi trung bình (giây)
- Token Cost: tổng số token sử dụng mỗi truy vấn
- Kết quả báo cáo dạng: trung bình $\pm$ độ lệch chuẩn

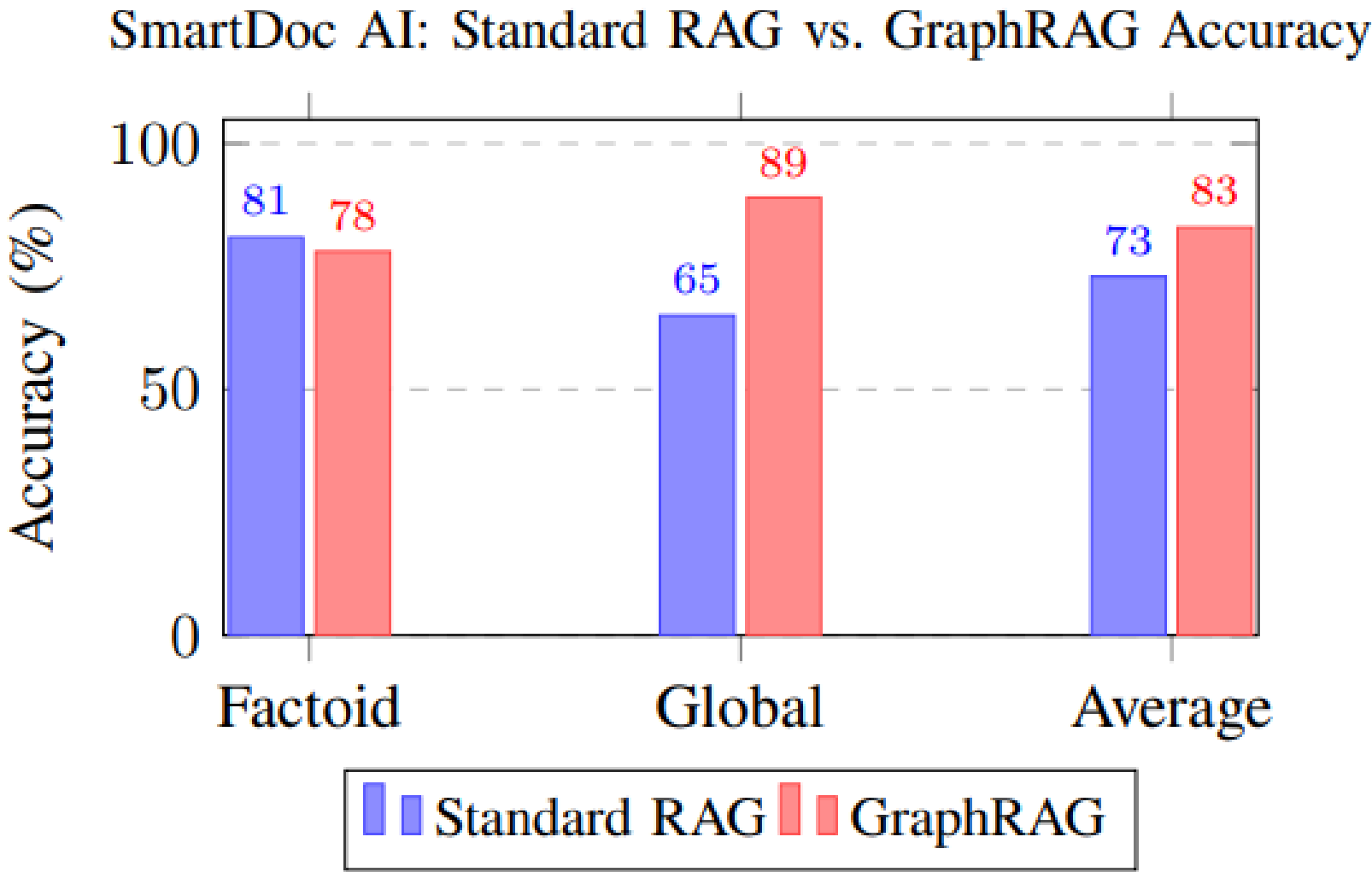
Kết quả định lượng

Bảng IV: So sánh hiệu năng trên SmartDoc AI ($n = 30$)

Chỉ số	Std RAG	GraphRAG
<i>Factoid Queries</i>		
Accuracy	81%	78%
Completeness	74%	71%
Latency	1.2 s	4.8 s
<i>Global Synthesis Queries</i>		
Accuracy	65%	89%
Completeness	62%	91%
Latency	1.5 s	12.3 s
<i>Tài nguyên</i>		
Token/query	850	3200
Indexing time	8 s	6–12 phút
Storage	45 MB	280 MB

- Truy vấn toàn cục (Synthesis):
Standard RAG nhỉnh hơn 3% nhưng nhanh hơn nhiều lần
- Truy vấn toàn cục (Synthesis):
GraphRAG vượt trội rõ rệt

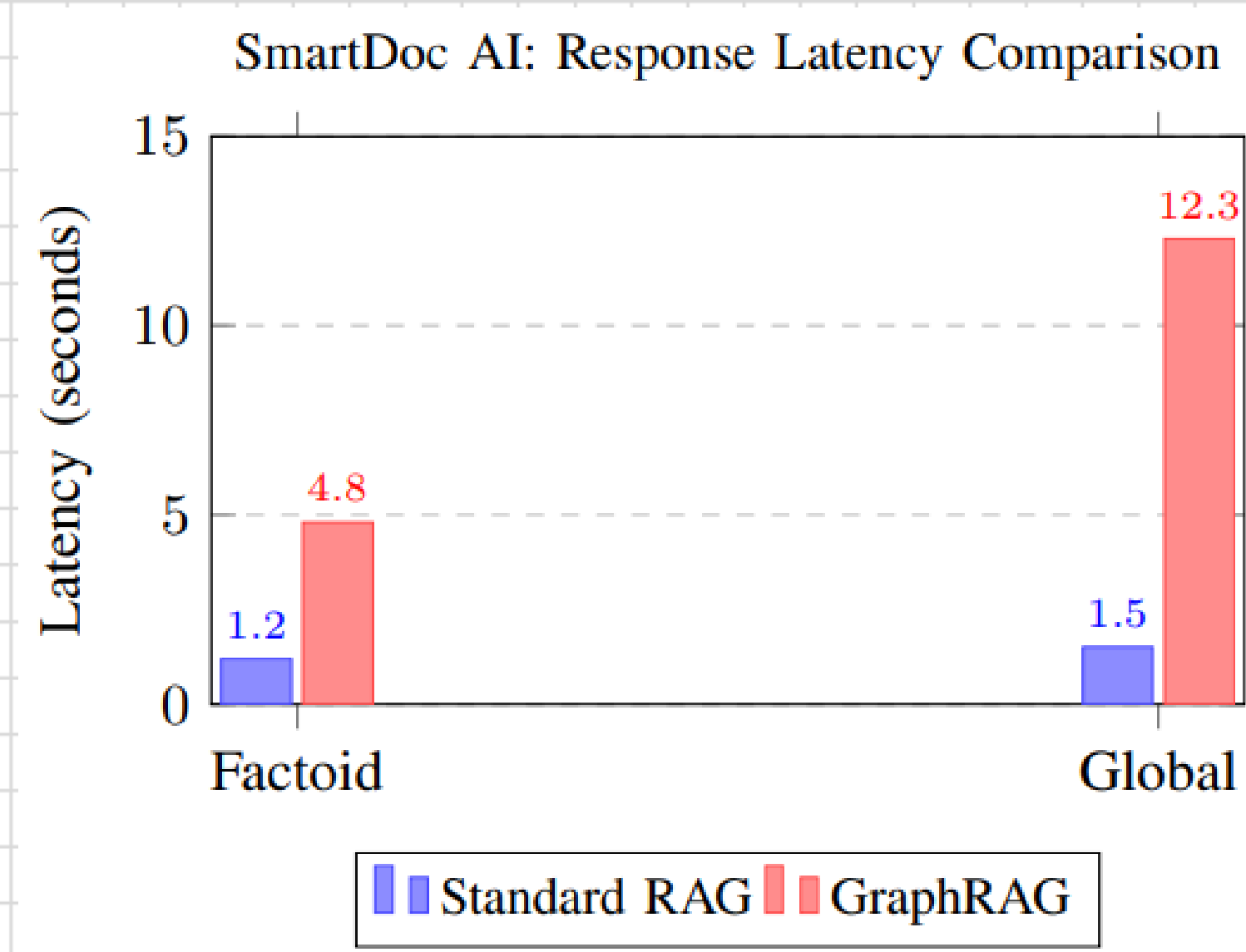
Biểu đồ độ chính xác



Loại truy vấn	Standard RAG	GraphRAG	Chênh lệch
Cụ thể (Factoid)	81%	78%	RAG tốt hơn +3%
Toàn cục (Synthesis)	65%	89%	GraphRAG tốt hơn +24%
Trung bình	73%	83%	GraphRAG tốt hơn +10%

- GraphRAG vượt trội trong tổng hợp toàn cục (+24 điểm %)
- Standard RAG tốt hơn một chút trong tra cứu dữ kiện cụ thể (+3 điểm %)

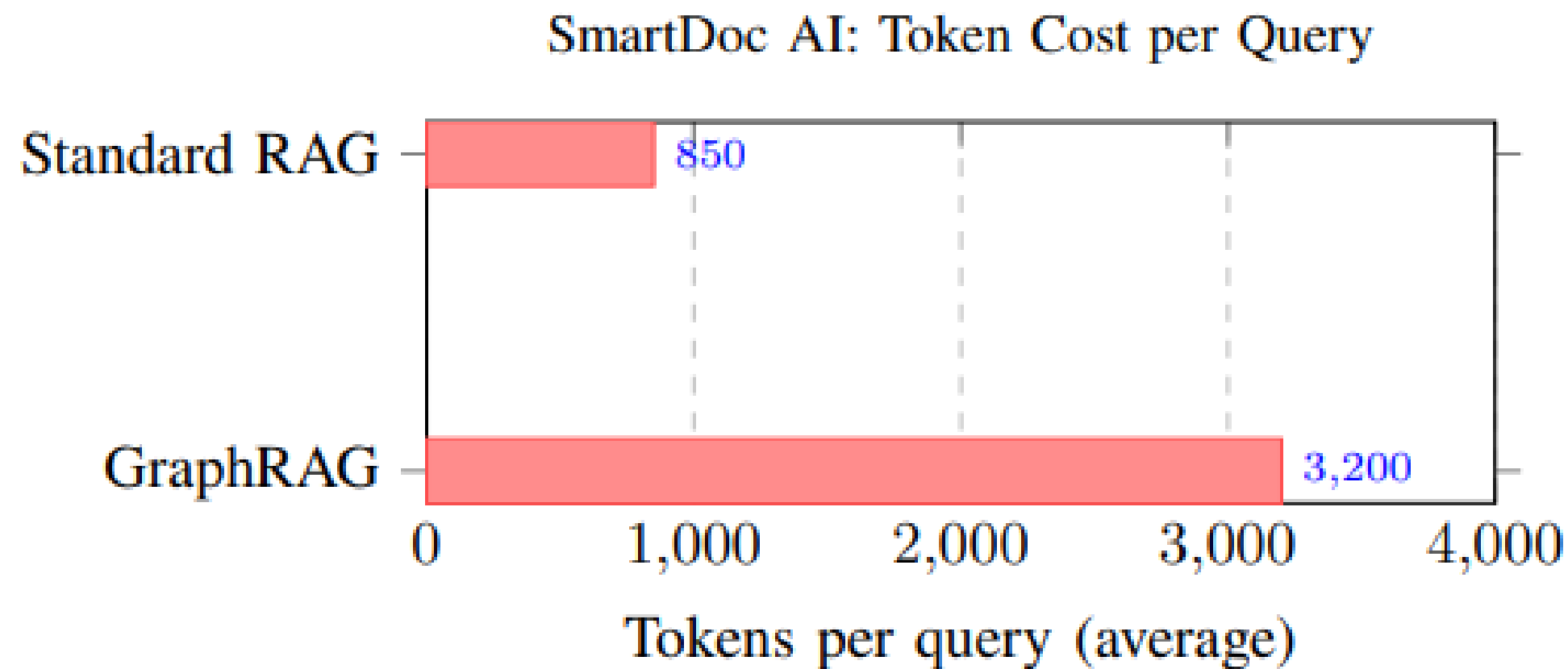
Biểu đồ độ trễ & chi phí token



Loại truy vấn	Standard RAG	GraphRAG	Chênh lệch
Cụ thể	1,2 s	4,8 s	RAG nhanh hơn 4×
Toàn cục	1,5 s	12,3 s	GRAPHRAG nhanh hơn 8,2×

Standard RAG: ưu thế tốc độ (nhanh hơn 4–8 lần) và chi phí (rẻ hơn 3,5 lần)

Biểu đồ độ trễ & chi phí token



Hình 4: So sánh chi phí token

- Standard RAG: 850 token
- GraphRAG: 3.200 token
- GraphRAG tiêu thụ gấp 3,76 lần — phản ánh chi phí xây dựng ngữ cảnh từ báo cáo cộng đồng và tóm tắt thực thể

- GraphRAG: ưu thế chất lượng (+24%) nhưng phải trả giá bằng tài nguyên

Phân tích định tính

Standard RAG

- Trích dẫn gần như nguyên văn các đoạn văn có điểm cosine cao nhất
- Điểm mạnh: chính xác khi tra cứu sự thật, định nghĩa, số liệu — câu trả lời dễ kiểm chứng nguồn gốc
- Điểm yếu: kém trong việc kết nối các khái niệm phân tán — đặc biệt thất bại khi câu hỏi đòi hỏi tổng hợp từ hơn hai vị trí trong tài liệu

GraphRAG:

- Trải dài trên nhiều phần của tài liệu, giải thích các mối liên hệ nhân quả
- Điểm mạnh: hỗ trợ suy luận đa bước, câu trả lời gắn kết và toàn diện hơn, kết nối các khái niệm phân tán
- Điểm yếu: đôi khi tổng quát hóa quá mức (over-generalization) → mất độ chính xác về số liệu hoặc định nghĩa cụ thể

Hạn chế của nghiên cứu

- Quy mô mẫu nhỏ ($n = 30$): kết luận cần được xác nhận thêm trên các tập dữ liệu lớn hơn và đa dạng hơn
- Dữ liệu chỉ giới hạn trong miền kỹ thuật: chưa kiểm chứng trên các miền khác (y tế, pháp lý, kinh doanh)
- GraphRAG phụ thuộc vào chất lượng trích xuất LLM: nếu LLM trích xuất thực thể kém $\rightarrow$ đồ thị tri thức sai lệch $\rightarrow$ kết quả sa sút toàn bộ
- Chi phí chưa được chuẩn hóa hoàn toàn: chi phí lập chỉ mục chưa được phân bổ vào chi phí mỗi truy vấn
- Chưa tích hợp framework đánh giá tự động: chưa dùng RAGAS [7] để đo faithfulness, answer relevancy, context precision — vẫn dùng đánh giá thủ công

